



Hochschule für angewandte Wissenschaften Coburg  
Fakultät Elektrotechnik und Informatik

Studiengang: Informatik

Bachelorarbeit

# Multi-Modal Feature Fusion with Cross-Attention for Tabular and Textual Data

Städler, Sebastian

Abgabe der Arbeit: 13.01.2026

Betreut durch:

Prof. Dr. Roman Rischke, Hochschule Coburg

## **Abstract**

TODO ...

# Inhaltsverzeichnis

|                                                                         |            |
|-------------------------------------------------------------------------|------------|
| <b>Abstract</b>                                                         | <b>I</b>   |
| <b>Abbildungsverzeichnis</b>                                            | <b>IV</b>  |
| <b>Tabellenverzeichnis</b>                                              | <b>V</b>   |
| <b>Codebeispielverzeichnis</b>                                          | <b>VI</b>  |
| <b>Abkürzungsverzeichnis</b>                                            | <b>VII</b> |
| <b>1 Einführung</b>                                                     | <b>1</b>   |
| <b>2 Theoretische Grundlagen</b>                                        | <b>2</b>   |
| 2.1 Multimodale Daten . . . . .                                         | 2          |
| 2.1.1 Heterogenität in multimodalen Daten . . . . .                     | 2          |
| 2.1.2 Repräsentation von Textdaten . . . . .                            | 2          |
| 2.1.3 Repräsentation tabellarischer Daten . . . . .                     | 4          |
| 2.2 Transformer-Architekturen und Attention-Mechanismen . . . . .       | 6          |
| 2.2.1 Der Transformer-Encoder . . . . .                                 | 6          |
| 2.2.2 Scaled Dot-Product Attention . . . . .                            | 7          |
| 2.2.3 Self-Attention vs. Cross-Attention . . . . .                      | 7          |
| 2.3 Strategien der multimodalen Fusion . . . . .                        | 8          |
| 2.3.1 Mechanismen der Fusion . . . . .                                  | 8          |
| 2.3.2 Architekturen und Positionierung der Fusion . . . . .             | 9          |
| <b>3 Verwandte Arbeiten</b>                                             | <b>11</b>  |
| 3.1 Cross-Attention in multimodalen Transformer-Architekturen . . . . . | 11         |
| 3.2 Verarbeitung von Tabellen und Text mit Transformern . . . . .       | 11         |
| 3.3 Forschungslücke und Einordnung . . . . .                            | 13         |
| <b>4 Methodik</b>                                                       | <b>14</b>  |
| 4.1 Datenbasis und Vorverarbeitung . . . . .                            | 14         |
| 4.2 Baseline-Modelle . . . . .                                          | 16         |
| 4.2.1 SumBERT (Early Fusion) . . . . .                                  | 16         |
| 4.2.2 ConcatBERT (Late Fusion) . . . . .                                | 17         |
| 4.3 CrossBERT-Architektur . . . . .                                     | 18         |
| 4.3.1 Architekturüberblick . . . . .                                    | 18         |
| 4.3.2 TabularTokenizer . . . . .                                        | 18         |
| 4.3.3 Cross-Attention-Block . . . . .                                   | 19         |

|                                                          |              |
|----------------------------------------------------------|--------------|
| 4.3.4 Fusionsvarianten: Early, Late und Hybrid . . . . . | 21           |
| 4.4 Trainings-Setup und Evaluationsmetriken . . . . .    | 21           |
| 4.5 Ablationsstudien . . . . .                           | 23           |
| <b>5 Ergebnisse</b>                                      | <b>24</b>    |
| <b>6 Diskussion und Fazit</b>                            | <b>25</b>    |
| <b>Literaturverzeichnis</b>                              | <b>VIII</b>  |
| <b>A1 Anhang</b>                                         | <b>XII</b>   |
| A1.1 Modellarchitektur CrossFitter . . . . .             | XII          |
| A1.2 Quellcode der Baseline-Modelle . . . . .            | XII          |
| A1.3 Quellcode der CrossBERT-Architektur . . . . .       | XVI          |
| <b>Ehrenwörtliche Erklärung</b>                          | <b>XVIII</b> |

## Abbildungsverzeichnis

|         |                                                                                                                                               |     |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Abb. 1: | Architektur von SumBERT: Addition der Tabellen-Repräsentation zum CLS-Token-Embedding vor dem Encoder. Quelle: [Fue24]. . . . .               | 16  |
| Abb. 2: | Architektur von ConcatBERT: Konkatenation der Tabellen-Features mit dem Output des BERT-Encoders. Quelle: [Fue24]. . . . .                    | 17  |
| Abb. 3: | Architektur von CrossBERT mit optionaler Early- und Late-Cross-Attention über Tab Tokens. . . . .                                             | 19  |
| Abb. 4: | Komponenten der CrossBERT-Architektur: Links die Projektion der Features in Tab Tokens, rechts deren Integration via Cross-Attention. . . . . | 20  |
| Abb. 5: | Schematische Darstellung der CrossFitter-Modellarchitektur im produktiven Einsatz. Quelle: [Fue24]. . . . .                                   | XII |

## Tabellenverzeichnis

|         |                                                                             |    |
|---------|-----------------------------------------------------------------------------|----|
| Tab. 1: | Exemplarischer Datensatz der internen Regressdatenbasis (anonymisiert). . . | 14 |
| Tab. 2: | Zentrale Trainingshyperparameter (Basiskonfiguration ohne Gating). . . .    | 22 |
| Tab. 3: | Übersicht der geplanten Ablationsstudien . . . . .                          | 23 |

## Codebeispielverzeichnis

|   |                                                              |     |
|---|--------------------------------------------------------------|-----|
| 1 | PyTorch-Implementierung von ConcatBERT und SumBERT . . . . . | XII |
| 2 | PyTorch-Implementierung von CrossBERT . . . . .              | XVI |

## Abkürzungsverzeichnis

|         |                                                         |
|---------|---------------------------------------------------------|
| AP      | Average Precision                                       |
| AUC-ROC | Area Under the Receiver Operating Characteristic Curve  |
| BERT    | Bidirectional Encoder Representations from Transformers |
| ELMo    | Embeddings from Language Models                         |
| FLOPs   | Floating Point Operations                               |
| GBDT    | Gradient Boosted Decision Tree                          |
| LGBM    | Light Gradient Boosting Machine                         |
| MLP     | Multi-Layer Perceptron                                  |
| NLP     | Natural Language Processing                             |
| OHE     | One-Hot Encoding                                        |
| TF-IDF  | Term Frequency-Inverse Document Frequency               |

# 1 Einführung

TODO ...

## 2 Theoretische Grundlagen

### 2.1 Multimodale Daten

Eine *Modalität* bezeichnet eine eigenständige Form der Informationsrepräsentation mit eigener Struktur (z. B. sequenzieller Text vs. tabellarische Attribute). Multimodales Lernen zielt darauf ab, diese verschiedenen Quellen gemeinsam zu verarbeiten, um ein umfassenderes Bild eines Sachverhalts zu gewinnen, als es durch eine einzelne Modalität möglich wäre [BAM19].

Zentral sind dabei zwei Aufgaben [L<sup>+</sup>24]: *Alignment* und *Fusion*. Alignment stellt die inhaltliche Beziehung zwischen den Modalitäten her (z. B. Zuordnung von Text zu Tabellenwerten), während Fusion die Informationen zu einer gemeinsamen Repräsentation verknüpft. Durch diese Kombination können sich die Modalitäten gegenseitig ergänzen und Wissenslücken schließen, die bei isolierter Betrachtung bestehen bleiben würden.

#### 2.1.1 Heterogenität in multimodalen Daten

Die Kombination von Text- und Tabellendaten stellt hierbei eine besondere Herausforderung dar. Während Text linguistisch strukturiert ist und Semantik aus der Reihenfolge von Tokens gewinnt, sind Tabellen attributbasiert und nicht-sequenziell. Dieser Unterschied, in der Literatur auch „Manifold Mismatch“ genannt, führt dazu, dass Text und Tabellen in sehr unterschiedlichen Zahlenräumen beschrieben werden [BAM19, HKCK20]. Text liegt in dichten Embedding-Vektoren, Tabellendaten in heterogenen, oft schlecht skalierten Merkmalen. Aus diesem Grund kann man Self-Attention, wie sie bei Text gut funktioniert, nicht einfach direkt auf rohe Tabellendaten anwenden.

Die zentrale Herausforderung besteht somit darin, diese strukturelle Lücke zu überbrücken. Bevor eine Fusion technisch möglich ist, müssen beide Modalitäten in einen kompatiblen, gemeinsamen Vektorraum projiziert werden. Dies erfordert allerdings spezifische Repräsentationsstrategien. Während für Text schon etablierte Tokenisierungsverfahren existieren, müssen für tabellarische Daten erst analoge Methoden der Tokenisierung angewendet werden, um sie in eine verarbeitbare Sequenzform zu überführen. Die nächsten Abschnitte beschreiben, wie Text (Abschnitt 2.1.2) und Tabellen (Abschnitt 2.1.3) in dieser Arbeit konkret dargestellt werden.

#### 2.1.2 Repräsentation von Textdaten

Um das Verständnis der nachfolgenden Architekturen und der später beschriebenen Tabular Tokenization zu erleichtern, werden zunächst die zentralen Begriffe Token, Repräsentation und Embedding als gemeinsame Grundlage definiert:

- **Token:** Ein Token stellt die kleinste diskrete Einheit einer Eingabesequenz dar, in die Rohdaten zerlegt werden, bevor sie ein Modell verarbeitet. Während dies im Textbereich Wörter oder Wortteile (Sub-words) sein können [DCLT19], werden in multimodalen Modellen auch Bild-Patches [DBK<sup>+</sup>21] als Tokens behandelt. Im Modell dienen Tokens als eindeutige Identifikatoren (Token IDs), die eine Verbindung zwischen Rohdaten und ihrer mathematischen Repräsentation (Vektoren) herstellen.
- **Repräsentation:** Unter Repräsentation wird allgemein jede numerische Kodierung (Vektorisierung) von Informationen verstanden, die es einem Modell erlaubt, Semantik, Struktur oder Merkmale von Daten mathematisch zu erfassen. Dazu zählen sowohl manuell definierte Merkmalsräume (z. B. Bag-of-Words oder Term Frequency-Inverse Document Frequency (TF-IDF)) als auch gelernte Vektoren [AU22].
- **Embedding:** Ein Embedding ist eine spezielle Form der Repräsentation: eine gelernte, dichte Vektorkodierung, deren Koordinaten als Modellparameter optimiert werden. Man unterscheidet zwischen statischen Einbettungen (z. B. Word2Vec [MCCD13]), die jedem Wort unabhängig vom Kontext einen festen Vektor zuordnen, und *kontextualisierten Embeddings* (z. B. Bidirectional Encoder Representations from Transformers (BERT) [DCLT19]), bei denen die Bedeutung eines Tokens dynamisch aus seinem umgebenden Kontext abgeleitet wird.

**Entwicklung der Textrepräsentation** Historisch wurden Textdaten zunächst mit frequenzbasierten Verfahren wie dem Bag-of-Words-Modell oder der TF-IDF beschrieben [SB88]. Diese Ansätze bilden Dokumente als Vektoren von Worthäufigkeiten ab, berücksichtigen aber kaum Grammatik oder semantische Beziehungen zwischen Wörtern und führen häufig zu hochdimensionalen, dünn besetzten Vektoren.

Einen wichtigen Schritt brachten verteilte Wortrepräsentationen wie Word Embeddings, insbesondere das Word2Vec-Verfahren [MCCD13]. Wörter werden dabei in einen dichten, niedrigdimensionalen Vektorraum projiziert, der durch einfache neuronale Netze gelernt wird. Semantisch ähnliche Wörter liegen in diesem Raum nah beieinander. Ein Nachteil dieser statischen Embeddings bleibt jedoch bestehen: Jedes Wort erhält unabhängig vom Kontext denselben Vektor, sodass Mehrdeutigkeiten nur eingeschränkt abgebildet werden können.

Kontextuelle Embeddings wie Embeddings from Language Models (ELMo) [PNI<sup>+</sup>18] und später BERT [DCLT19] lösen dieses Problem, weil sie Wortvektoren abhängig vom jeweiligen Satzkontext berechnen. BERT wird als Encoder-only-Modell auf großen Textmengen vortrainiert und nutzt dabei die Transformer-Architektur sowie Self-Attention-Mechanismen (siehe Abschnitt 2.2), um für jedes Token eine kontextabhängige Repräsentation zu erzeugen.

Für die Verarbeitung ganzer Sätze oder Dokumente, wie sie in dieser Arbeit relevant ist, stoßen reine Token-Embeddings jedoch an Grenzen. Hier kommen Techniken wie Pooling oder die Verwendung spezieller Tokens ins Spiel. Insbesondere das  $[CLS]$ -Token (Classifier Token) in Modellen wie BERT wird jeder Eingabesequenz vorangestellt, sammelt über Self-Attention kontextuelle Informationen und sein finaler Vektor  $T_{[CLS]}$  dient als kompakte Satzrepräsentation für Klassifikationsaufgaben [DCLT19]. In dieser Arbeit wird das  $[CLS]$ -Token als kompakte Textrepräsentation genutzt, um Textinformationen später mit tabellarischen Merkmalen zu kombinieren.

### 2.1.3 Repräsentation tabellarischer Daten

Tabellarische Daten bilden die Grundlage zahlreicher Anwendungen in der Industrie, stellen jedoch für Deep-Learning-Ansätze eine besondere Herausforderung dar [BLS<sup>+</sup>24]. Um die in Abschnitt 2.1.1 beschriebene Lücke zu schließen, ist eine Transformation der rohen Tabellendaten in eine kompatible Vektor-Repräsentation notwendig.

**Herausforderungen der Tabellenstruktur** Tabellen enthalten unterschiedliche Arten von Merkmalen, z. B. numerische Größen wie Preis oder Alter und kategoriale Angaben wie Produkt oder Postleitzahl [BLS<sup>+</sup>24, BSP23].

Ein weiteres wichtiges Merkmal ist die Permutationsinvarianz von Spalten. Anders als Wörter in einem Satz haben die Spalten einer Tabelle keine natürliche Ordnung. Idealerweise sollte ein Modell daher robust gegenüber Vertauschungen der Feature Positionen sein, solange die Zuordnung von Wert und Feature Typ erhalten bleibt [GRKB21]. Viele praktische Deep Learning Modelle für Tabellendaten, einschließlich der in dieser Arbeit untersuchten, erfüllen dies nicht explizit und setzen eine feste Spaltenreihenfolge zwischen Training und Inferenz voraus. Im Unterschied zu Textsequenzen mit Tokenisierung verändert gleiche Permutation der Spalten in beiden Phasen die Bedeutung der tabellarischen Features jedoch nicht.

**Grenzen klassischer Vorverarbeitungsmethoden** In traditionellen Machine-Learning-Verfahren, wie Gradient Boosted Decision Trees (GBDTs), werden kategoriale Daten häufig mittels One-Hot Encoding (OHE) verarbeitet. Dabei wird eine kategoriale Variable mit der Kardinalität  $C$  in einen binären Vektor der Länge  $C$  transformiert. Für Deep-Learning-Modelle und insbesondere für die Integration in Transformer-Architekturen ist OHE jedoch oft unpraktisch: Die Vektoren werden sehr lang und enthalten fast nur Nullen [HKCK20, GRKB21]. Außerdem macht OHE keine inhaltlichen Beziehungen zwischen Kategorien sichtbar. Alle Kategorien werden als orthogonal betrachtet, das heißt, der Abstand zwischen allen Kategorien im Vektorraum ist identisch. Semantische Beziehungen, wie etwa die Ähnlichkeit zwischen den Kategorien „Auto“ und „LKW“ im

Vergleich zu „Apfel“, können durch OHE nicht abgebildet werden [HKCK20]. In dieser Arbeit wird OHE für kategoriale Merkmale trotzdem als einfache, verlustfreie Kodierung der Rohdaten genutzt. Die Verdichtung in einen sinnvollen latenten Raum übernimmt eine nachgelagerte Projektionsschicht, die aus den dünn besetzten Eingaben eine dichte Repräsentation lernt.

Auch einfache Multi-Layer Perceptrons (MLPs) stoßen auf rohen Daten an ihre Grenzen, da sie numerische Eingaben lediglich als skalare Werte betrachten und im ersten Schritt keine feature-spezifische Verarbeitung ermöglichen, wie sie beispielsweise für das Erlernen von Interaktionen zwischen spezifischen Features notwendig wäre [GRKB21].

**Tabular Tokenization und Latenter Raum** Der Prozess, tabellarische Daten in eine für Transformer verarbeitbare Sequenz zu überführen, wird im Folgenden als *Tabular Tokenization* bezeichnet [BLS<sup>+</sup>24, BSP23]. In dieser Arbeit bedeutet dies, dass eine Tabellenzeile  $x$  in eine kurze Folge von Vektoren umgewandelt wird, wie sie ein Sprachmodell erwartet:

$$\mathbf{E} = [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N] \quad \text{mit} \quad \mathbf{e}_i \in \mathbb{R}^H \quad (2.1)$$

Hierbei entspricht  $H$  der Dimension des Hidden-States des verwendeten Transformer-Modells (z. B.  $H = 768$  bei BERT). Durch diese Angleichung der Dimensionen kann später Cross-Attention zwischen Text- und Tabellen-Tokens angewendet werden [BSP23]. Im Gegensatz zu OHE erlaubt dieser dichte Vektorraum das Erlernen semantischer Ähnlichkeiten, sodass verwandte Features im Vektorraum geometrisch näher beieinander liegen.

Es ist jedoch essenziell zu unterscheiden: Während bei Text-Tokens die Reihenfolge die semantische Bedeutung bestimmt, handelt es sich bei *Tabular Tokens* um eine künstlich erzeugte Sequenz ohne natürliche Ordnung. Die Sequenzform dient hier rein als technisches Konstrukt für die Attention-Verarbeitung, nicht als Träger syntaktischer Informationen.

**Architekturen der Repräsentation** Hinsichtlich der Struktur der erzeugten Sequenz lassen sich zwei Hauptstrategien unterscheiden, die den Unterschied zwischen spaltenweiser und zeilenweiser Verarbeitung verdeutlichen [BSP23]:

Der erste Ansatz, bekannt als **Feature Tokenization** (oder Column-wise Embeddings), bildet jedes Feature der Ursprungstabelle isoliert auf genau einen Token-Vektor ab. Dies wird beispielsweise im *TabTransformer* [HKCK20] oder im *SAINT*-Modell [SGS<sup>+</sup>21] angewendet. Die Sequenzlänge  $N$  entspricht hierbei exakt der Anzahl der Spalten. Der Vorteil liegt in der Erhaltung der feingranularen Information jedes einzelnen Features.

Der zweite Ansatz, der in dieser Arbeit verfolgt wird, basiert auf einer **globalen Projektion** (Row-level Tokenization). Anstatt jedes Feature einzeln zu tokenisieren, werden hierbei alle Merkmale einer Tabellenzeile zunächst konkateniert und durch ein MLP in eine latente Reprä-

sensation überführt [GB21, BLS<sup>+</sup>24]. Diese Repräsentation besteht aus einer fixen Anzahl an Vektoren, wodurch die Sequenzlänge des Transformers von der Anzahl der Eingabefeatures entkoppelt wird. Ein wesentlicher Vorteil dieses Verfahrens ist, dass Zusammenhänge zwischen den Features bereits in der Projektionsphase erfasst werden können und die Rechenkomplexität der Attention-Schicht kontrollierbar bleibt.

Unabhängig von der gewählten Methode entsteht am Ende eine Sequenz dichter Vektoren. Erst dadurch können Transformer-Modelle die Tabellendaten mit denselben Operationen verarbeiten wie Text und sie anschließend über Mechanismen wie Cross-Attention mit Textrepräsentationen verknüpfen.

## 2.2 Transformer-Architekturen und Attention-Mechanismen

Nachdem im vorangegangenen Abschnitt die Grundlagen der Datenrepräsentation für Text und Tabellen erläutert wurden, geht es im nächsten Schritt um die Transformer-Architektur, die heute in vielen Natural Language Processing (NLP)-Anwendungen eingesetzt wird. Das Verständnis der Attention-Mechanismen ist wichtig, um die später verwendete multimodale Fusion einordnen zu können. Ursprünglich wurde der Transformer für maschinelle Übersetzungsaufgaben konzipiert, hat sich aber als Standard für die Modellierung komplexer Abhängigkeiten in sequenziellen Daten etabliert [VSP<sup>+</sup>17].

### 2.2.1 Der Transformer-Encoder

Der *Transformer* wurde 2017 von Vaswani et al. vorgestellt [VSP<sup>+</sup>17]. Im Gegensatz zu früheren Modellarchitekturen, die primär auf Rekurrenz (z. B. RNNs, LSTMs) oder Faltung (CNNs) basierten, verzichtet der Transformer vollständig auf diese Mechanismen und nutzt stattdessen *Self-Attention* als zentrales Element. Die ursprüngliche Architektur besteht aus zwei Hauptkomponenten: einem Encoder, der die Eingabesequenz verarbeitet und in eine abstrakte Repräsentation überführt, und einem Decoder, der basierend darauf eine Ausgabesequenz generiert.

Für die Aufgabenstellung dieser Arbeit, bei der es um die Klassifikation von multimodalen Daten geht, ist vor allem der Encoder-Teil relevant. Encoder-only-Modelle, wie das in Abschnitt 2.1.2 vorgestellte BERT, nutzen einen Stapel von Transformer-Blöcken, um für jedes Eingabe-Token einen kontextualisierten Vektor zu berechnen [DCLT19]. Im Gegensatz zu statischen Embeddings, bei denen ein Wort stets denselben Vektor erhält, integriert der Transformer-Encoder Informationen aus dem gesamten Kontext der Sequenz in die Repräsentation jedes einzelnen Tokens und dient damit als leistungsfähiger Feature-Extraktor.

### 2.2.2 Scaled Dot-Product Attention

Das zentrale Element eines jeden Transformer-Blocks ist der Attention-Mechanismus. Er ermöglicht es dem Modell, die Relevanz verschiedener Teile der Eingabe für den aktuellen Verarbeitungsschritt dynamisch zu gewichten. Vaswani et al. formalisieren dies als *Scaled Dot-Product Attention* [VSP<sup>+</sup>17].

Vereinfacht gesagt vergleicht das Modell eine Abfrage (*Query*  $\mathbf{Q}$ ) mit vielen möglichen Kontextstellen (*Keys*  $\mathbf{K}$ ) und bildet dann eine gewichtete Summe der zugehörigen Werte (*Values*  $\mathbf{V}$ ). Tokens, die besser zur Query passen, erhalten höhere Gewichte. Mathematisch wird dies durch folgende Gleichung ausgedrückt:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (2.2)$$

Hierbei berechnet das Skalarprodukt  $\mathbf{Q}\mathbf{K}^T$  die Ähnlichkeit zwischen der Query und den Keys. Der Faktor  $\frac{1}{\sqrt{d_k}}$  dient der Skalierung, um numerische Instabilitäten bei großen Dimensionen zu vermeiden [VSP<sup>+</sup>17]. Die Softmax-Funktion normalisiert diese Ähnlichkeitswerte zu Wahrscheinlichkeiten, die sich zu 1 aufsummieren. Schließlich wird eine gewichtete Summe der Values  $\mathbf{V}$  gebildet.

### 2.2.3 Self-Attention vs. Cross-Attention

Ein wesentlicher Aspekt moderner Transformer-Architekturen und insbesondere multimodaler Modelle ist die Definition der Quellen für die Vektoren  $\mathbf{Q}$ ,  $\mathbf{K}$  und  $\mathbf{V}$ . Hierbei wird zwischen *Self-Attention* und *Cross-Attention* unterschieden.

**Self-Attention (Intra-Modal)** Im Standard-Encoder, wie er von Vaswani et al. [VSP<sup>+</sup>17] beschrieben wird, stammen Query, Key und Value aus derselben Eingabequelle (z. B. der Ausgabe des vorherigen Layers). Dies wird als Self-Attention bezeichnet.

- **Funktionsweise:** Jedes Token der Sequenz berechnet seine Aufmerksamkeit in Bezug auf alle anderen Tokens derselben Sequenz.
- **Zweck:** Modellierung von Abhängigkeiten innerhalb einer Modalität. Dies erlaubt dem Modell, kontextuelle Beziehungen, wie etwa Referenzen innerhalb eines Satzes, zu erfassen.

**Cross-Attention (Inter-Modal)** Für die Fusion unterschiedlicher Modalitäten, wie Text und Tabellendaten, ist die Cross-Attention von zentraler Bedeutung. Dieses Konzept findet sich in diversen multimodalen Architekturen [TB19].

- **Funktionsweise:** Die Vektoren werden aus unterschiedlichen Quellen gespeist. Beispielsweise generiert Modalität A (z. B. Text) die Queries  $\mathbf{Q}$ , während Modalität B (z. B. Tabelle) die Keys  $\mathbf{K}$  und Values  $\mathbf{V}$  bereitstellt.
- **Zweck:** Ermöglichung eines Informationsflusses über Modalitätsgrenzen hinweg. Die Text-Repräsentationen können gezielt Informationen aus den Tabellen-Features integrieren, die für den aktuellen Kontext relevant sind. Dies bildet das theoretische Fundament für flexible Fusions-Architekturen, da es eine dynamische Verknüpfung heterogener Datenräume erlaubt.

### 2.3 Strategien der multimodalen Fusion

Voraussetzung für jede Fusion ist, dass die unterschiedlichen Modalitäten bereits in einen kompatiblen Vektorraum transformiert wurden (vgl. Abschnitt 2.1). Die Strategien der multimodalen Fusion befassen sich anschließend mit der Frage, *wie* (Mechanismen) und *wann* (Architekturen) diese Informationen technisch kombiniert werden [L<sup>+</sup>24, BAM19].

#### 2.3.1 Mechanismen der Fusion

Die technische Realisierung der Fusion, also *wie* die Vektoren der unterschiedlichen Modalitäten zusammengeführt werden, kann durch verschiedene mathematische Operationen erfolgen [J<sup>+</sup>24, BAM19]. Ein zentrales Konzept ist hierbei die *Joint Representation*. Das Ziel ist es, die unimodalen Eingangsvektoren in einen gemeinsamen semantischen Unterraum zu projizieren, um eine einzige, multimodale Repräsentation  $\mathbf{Z}$  zu erzeugen [J<sup>+</sup>24, L<sup>+</sup>24]. Mathematisch lässt sich dieser Vorgang allgemein formulieren als:

$$\mathbf{Z} = f(\mathbf{X}_{\text{text}}, \mathbf{X}_{\text{tab}}) \quad (2.3)$$

Wobei  $f$  eine Funktion darstellt (beispielsweise ein neuronales Netz oder eine deterministische Operation), die aus den unimodalen Repräsentationen  $\mathbf{X}_{\text{text}}$  und  $\mathbf{X}_{\text{tab}}$  die fusionierte multimodale Repräsentation  $\mathbf{Z}$  berechnet [BAM19].

Zu den gängigsten spezifischen Operationen zählen:

**Concatenation (Verkettung)** Dies ist die wohl einfachste Form der Fusion, bei der die Merkmalsvektoren lediglich hintereinander gehängt werden:

$$\mathbf{Z} = [\mathbf{X}_{\text{text}} \parallel \mathbf{X}_{\text{tab}}] \quad (2.4)$$

Ein wesentlicher Vorteil ist, dass die Eingangsvektoren  $\mathbf{X}_{\text{text}}$  und  $\mathbf{X}_{\text{tab}}$  nicht dieselbe Dimension besitzen müssen. Die Dimension des Ergebnisvektors  $\mathbf{Z}$  entspricht der Summe der Einzeldimensionen.

**Addition (Summierung)** Hierbei werden die Vektoren elementweise addiert:

$$\mathbf{Z} = \mathbf{X}_{\text{text}} + \mathbf{X}_{\text{tab}} \quad (2.5)$$

Dies setzt zwingend voraus, dass beide Vektoren dieselbe Dimension aufweisen, weshalb oft eine vorherige Projektion notwendig ist.

**Attention-Mechanismen** Im Gegensatz zu statischen Operationen wie der Addition oder Verkettung erlauben Attention-Mechanismen eine dynamische Gewichtung der Features. Besonders relevant ist die *Cross-Attention*. Hierbei können Repräsentationen einer Zielmodalität (z. B. Text) gezielt Informationen aus einer Quellmodalität (z. B. Tabelle) abfragen. Typischerweise stammen dabei die *Queries* aus der Zielmodalität, während *Keys* und *Values* aus der Quellmodalität geliefert werden. Dies ermöglicht eine gewichtete Aggregation von Informationen, die auf den aktuellen Kontext konditioniert ist, und erlaubt die Modellierung komplexer Abhängigkeiten über Modalitätsgrenzen hinweg [L<sup>+</sup>24].

### 2.3.2 Architekturen und Positionierung der Fusion

Neben der gewählten Operation ist wichtig, *an welcher Stelle* im Modell die Fusion stattfindet (früh, spät oder mehrfach in der Tiefe) [J<sup>+</sup>24, L<sup>+</sup>24, BAM19].

**Early Fusion (Daten- und Feature-Ebene)** Bei der Early Fusion werden die Modalitäten auf Rohdaten- oder Feature-Ebene zu einer gemeinsamen Eingabe kombiniert und anschließend in einem gemeinsamen Modellzweig (*One-Tower*) weiterverarbeitet [J<sup>+</sup>24, L<sup>+</sup>24, BAM19]. Dadurch können Interaktionen zwischen Merkmalen früh und kontextübergreifend gelernt werden, jedoch steigen die Anforderungen an Ausrichtung und Kompatibilität der Modalitäten; bei stark heterogenen Eingaben kann dies die Modellierung erschweren [BAM19, J<sup>+</sup>24].

**Late Fusion (Entscheidungs- und Ausgabe-Ebene)** Bei der Late Fusion werden Text und Tabelle zunächst in getrennten Zweigen (*Two-Tower*) verarbeitet und erst am Ende über ihre High-Level-Repräsentationen oder Prädiktionen kombiniert [L<sup>+</sup>24]. Typische Operatoren sind Mittelwertbildung, Voting oder ein nachgeschaltetes MLP [BAM19]. Der Ansatz ist modular und robust gegenüber fehlenden Modalitäten, modelliert jedoch kaum niedrigstufige Interaktionen; zudem wirken sich starke Störungen einer Modalität direkt auf die Endentscheidung aus [BAM19].

**Hybrid und Deep Fusion** Hybride Ansätze verbinden frühe und späte Fusion, indem sie zusätzliche Fusionsschichten zwischen getrennten Zweigen einfügen [J<sup>+</sup>24, BAM19, L<sup>+</sup>24]. In einigen Transformer-Modellen wird die Fusion über mehrere Layer hinweg wiederholt, häufig über Cross-Attention, sodass Interaktionen schrittweise verfeinert werden können [J<sup>+</sup>24, L<sup>+</sup>24].

## 3 Verwandte Arbeiten

Das Feld des multimodalen Lernens hat sich durch den Einsatz von Transformer-Modellen stark weiterentwickelt. Für Bild und Text gibt es inzwischen etablierte Verfahren auf Basis von Cross-Attention. Die gemeinsame Verarbeitung von tabellarischen Daten und Text ist dagegen noch relativ neu, und es gibt bisher keine klar dominierende Architektur. Dieses Kapitel ordnet die Arbeit in den aktuellen Forschungsstand ein, differenziert zwischen generellen Architekturen und spezifischen Ansätzen für Tabellen und stellt den im Unternehmenskontext relevanten Benchmark vor.

### 3.1 Cross-Attention in multimodalen Transformer-Architekturen

Cross-Attention ist ein wichtiger Baustein, um Informationen aus verschiedenen Datenquellen in Transformer-Modellen zusammenzubringen. Ein Modell kann damit gezielt Informationen einer Modalität in den Kontext einer anderen Modalität einbeziehen, ohne die ursprüngliche Struktur der Eingaben zu früh zu vermischen.

Im Bereich Vision Language zeigt **LXMERT** [TB19], wie eine Dual-Stream-Architektur genutzt werden kann. Bild und Text werden zunächst getrennt encodiert. Erst danach führt ein zusätzlicher Cross-Modality-Encoder beide Stränge zusammen. So bleiben mehr Informationen erhalten, als wenn man Bild- und Textdaten direkt am Eingang einfach aneinanderhängt.

**Flamingo** [ADL<sup>+</sup>22] geht noch einen Schritt weiter. Das Modell richtet sich auf Few-Shot-Szenarien und verwendet ein vortrainiertes Sprachmodell, das eingefroren bleibt. Visuelle Informationen werden über *Gated-Cross-Attention-Dense*-Schichten in dieses Sprachmodell eingespeist. Das zeigt, dass man Cross-Attention auch nachträglich an ein bestehendes Sprachmodell anbinden kann. Neuere Ansätze wie **PaLI** [C<sup>+</sup>23] bestätigen, dass solche dynamischen Interaktionen statischen Fusionsverfahren überlegen sind, wenn komplexe semantische Zusammenhänge zwischen Modalitäten modelliert werden sollen.

### 3.2 Verarbeitung von Tabellen und Text mit Transformern

Die Übertragung dieser Ideen auf Tabellen ist schwieriger, da tabellarische Daten im Gegensatz zu Text keine natürliche Reihenfolge der Einträge besitzen und heterogene Spaltentypen kombinieren (siehe Abschnitt 2.1.1).

Der **TabTransformer** [HKCK20] zeigt, dass Transformer Encoder auch für rein tabellarische, insbesondere kategoriale, Daten geeignet sind. Jede Spalte wird dabei als eigener Token betrachtet (Feature Tokenization), und Self Attention modelliert Abhängigkeiten zwischen den Spalten.

Obwohl TabTransformer keine Textdaten einbezieht, bildet er die Grundlage für spätere Arbeiten mit spaltenweiser Attention.

Für die Kombination von Tabellen und Text wählt **TaBERT** [YNYR20] einen anderen Weg. Tabellenzeilen werden in textähnliche Sequenzen linearisiert und gemeinsam mit natürlichsprachlichen Beschreibungen in ein BERT-Modell gegeben. Das entspricht einer frühen Fusion: Tabelle und Text werden schon auf Eingabeebene zu einer gemeinsamen Sequenz zusammengeführt. Die ursprüngliche Tabellenstruktur ist dann nur noch indirekt erkennbar.

Arbeiten wie das **Multimodal-Toolkit** [GB21] und Bonnier et al. [B<sup>+</sup>24] vergleichen verschiedene Architekturen. Bonnier et al. zeigen, dass spezialisierte multimodale Transformer in der Praxis oft nur geringe Vorteile gegenüber einfachen Baselines bringen, etwa gegenüber der Konkatination von Text- und Tabellen-Features. Ein wichtiger Einflussfaktor ist dabei die Qualität der einzelnen Modalitäten.

**Industrieller Benchmark: CrossFitter Stacking** Im Unternehmenskontext dieser Arbeit dient die interne Modellarchitektur „CrossFitter“ als ebenfalls relevante Baseline. Diese Architektur wurde im Rahmen eines Praktikums bei der Firma QuantCo entwickelt und in einer internen Präsentation vorgestellt [Fue24]. Im Gegensatz zu den zuvor beschriebenen End-to-End-Architekturen handelt es sich um einen zweistufigen Stacking-Ansatz, der die Stärken spezialisierter Modelle kombiniert, ohne dass diese in einem gemeinsamen, durchgängig trainierbaren Modell vereint sind. Abbildung 5 im Anhang A1 zeigt den schematischen Aufbau der CrossFitter-Modellarchitektur, basierend auf der internen Dokumentation [Fue24]. In den Experimenten dieser Arbeit wird CrossFitter jedoch nicht als direkte Vergleichsbaseline genutzt, da der produktive Stacking-Ansatz einem anderen Modellierungsprinzip folgt als die untersuchten End-to-End-Transformer-Modelle und die zugrunde liegende Pipeline nicht vollständig reproduzierbar ist.

Das Verfahren nutzt ein **2-Fold-Cross-Prediction-Schema**, um Textinformationen für ein tabellarisches Modell nutzbar zu machen:

1. Die Trainingsdaten werden in zwei Folds A und B aufgeteilt.
2. Ein BERT-Modell wird auf Fold A trainiert und erzeugt Vorhersagen für Fold B, also Out-of-Sample Predictions.
3. Analog wird ein zweites BERT-Modell auf Fold B trainiert und sagt die Fälle in Fold A voraus.

Die so gewonnenen Wahrscheinlichkeitswerte des Textmodells werden als zusätzliches Feature zusammen mit den ursprünglichen strukturierten Merkmalen in eine Light Gradient Boosting

Machine (LGBM) eingespeist. In der Praxis ist dieser Ansatz robust und leistungsfähig. Aus theoretischer Sicht bleibt jedoch eine Einschränkung bestehen, da die Modalitäten während des Trainings nicht direkt miteinander interagieren. Das Textmodell erhält keinen Rückfluss aus den Tabellendaten und kann seine Repräsentationen daher nicht an den Kontext der strukturierten Attribute anpassen.

### 3.3 Forschungslücke und Einordnung

In den betrachteten Arbeiten zeigt sich ein deutliches Bild:

- Für Bild und Text gibt es ausgereifte Cross-Attention-Architekturen (z. B. Flamingo).
- Für Tabelle und Text dominieren bisher Linearisierungsansätze (z. B. TaBERT) oder alternative Architekturen wie das vorgestellte Stacking-Verfahren (z. B. CrossFitter).

Für Tabular-Text-Kombinationen ist bisher nur begrenzt untersucht, wann Cross-Attention gegenüber einfachen Fusionsoperationen tatsächlich einen Mehrwert bringt und wie man solche Mechanismen sinnvoll im Modell platziert. Diese Arbeit greift diese Fragen auf und untersucht exemplarisch Cross-Attention-basierte Architekturen für Versicherungsdaten.

## 4 Methodik

### 4.1 Datenbasis und Vorverarbeitung

Die Datengrundlage besteht aus einer Mischung aus produktiven Versicherungsdaten und öffentlichen Benchmarks. Alle Datensätze werden über eine einheitliche Pipeline in Tensoren überführt (inkl. Splitting, tabellarischer Vorverarbeitung und Text-Tokenisierung).

**Datenbasis** Der primäre Datensatz stammt aus einem deutschen Versicherungsunternehmen und wurde für die Identifikation von Regresspotenzialen aufbereitet. Es handelt sich um ein Multi-Class-Setting mit den Zielklassen *Kein Regress* (0), *Regress* (1) und *Versicherungsschutzentzug* (2). Er kombiniert unstrukturierte Schadenbeschreibungen mit tabellarischen Merkmalen zum Schadenfall (Tabelle 1 zeigt einen exemplarischen Datenpunkt). Die Daten bergen jedoch eine Problematik: Es wurde historisch nur ein Teil der Fälle für eine ausführliche Regressprüfung ausgewählt, was zu einem *Selection Bias* in den Labels führt. Die nicht geprüften Daten enthalten wahrscheinlich noch unerkannte Regressfälle, auch wenn davon ausgegangen wird, dass dieses Potenzial gering ist (*Missing-Label-Problem*). Ungelabelte Fälle werden daher als Klasse 0 behandelt. Diese Problematiken (Bias und Missing Labels) sind nicht der Hauptfokus dieser Arbeit, werden aber der Vollständigkeit halber erwähnt, um die Eigenschaften des primären Datensatzes transparent zu machen.

Tab. 1: Exemplarischer Datensatz der internen Regressdatenbasis (anonymisiert).

|                               |                                                                                                                                                                                                                                                                                                         |  |  |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| <b>Metadaten</b>              | ID: 68443XXXX, Label: 1 (Regress), Status: Validiert (True)                                                                                                                                                                                                                                             |  |  |
| <b>Textmodalität</b>          | <i>Schadenhergang</i> : VN: Kreuzung: Vn hatte grünen Pfeil der aufgeleuchtet sei. Es kam zur Kollision Vorgang strittig. <i>Dokumente</i> : ausführliche HSA, strittige Ampel, PUM, EA-Geb-re io. <i>Notizen</i> : PWS empfiehlt Beauftragung eines SV; ADAC bereits auf dem Weg; wst wartet auf fg... |  |  |
| <b>Tabellarische Merkmale</b> | log_zlg_fzg: 9,839, kh_other_partially_paid: 1, untersparte: KF.VK, risikoschluessel: UNF, svorg_schadenursache: Unfall ein weiteres Kfz, had_ever_erinnerungsregress: 1                                                                                                                                |  |  |

Zur externen Validierung werden zwei Kaggle-Benchmarks genutzt:

1. **Women's E-Commerce Clothing Reviews** [Kag18]: Binäre Sentiment-Klassifikation mit Text und Metadaten (23.486 Samples gesamt; 21.137 Training, 2.349 Validierung).
2. **PetFinder.my Adoption Prediction** [Kag19]: Multi-Class-Klassifikation mit Beschreibungen und Metadaten (14.993 Samples gesamt; 13.494 Training, 1.499 Validierung).

Damit wird die Übertragbarkeit der Architektur außerhalb der Versicherungsdomäne überprüft.

**Datenvorverarbeitung** Die Vorverarbeitung trennt strikt Trainings-, Validierungs- und Testdaten, um Data Leakage zu vermeiden. Das Splitting erfolgt stratifiziert (bei Multi-Class) und verwendet, sofern verfügbar, einen vordefinierten Testsplit. Zusätzlich wird aus dem Trainingssplit ein Validierungsanteil (`validation_fraction`) abgeleitet. Optional wird die Klassenverteilung im Training über Undersampling der Majoritätsklasse auf ein vorgegebenes Verhältnis  $N(0)/N(1)$  (`train_imbalance_0_to_1`) eingestellt. Im Wesentlichen umfasst die Vorverarbeitung:

- **Tabellarische Daten:** OHE für kategoriale Merkmale (fehlende Werte als Kategorie `None`, unbekannte Kategorien werden ignoriert); Min-Max-Skalierung numerischer Variablen auf  $[0, 1]$ . Die Encoder/Scaler werden ausschließlich auf Trainingsdaten trainiert (*fit*) und auf Validierung/Test angewandt (*transform*).
- **Textdaten:** Tokenisierung mit `deepset/gbert-base` (inkl. `[CLS]` und `[SEP]`); Padding/Truncation auf die modellabhängige maximale Kontextlänge (für BERT: 512 Tokens).
- **Splitting und Balancing:** Stratified Split unter Beibehaltung der Klassenverteilung (insbesondere relevant bei Multi-Class, z. B. PetFinder). Im Training wird optional ein Random Undersampling der Majoritätsklasse eingesetzt, gesteuert über `train_imbalance_0_to_1`.

## 4.2 Baseline-Modelle

Um den Mehrwert der entwickelten CrossBERT-Architektur gegenüber bestehenden multimodalen Modellen einordnen zu können, erfolgt ein Vergleich mit zwei bestehenden multimodalen Modellen für die Fusion von tabellarischen Daten und Textdaten: SumBERT (Early Fusion) und ConcatBERT (Late Fusion). Die Konzepte, der Quellcode sowie die Architekturgrafiken (Abbildungen 1 und 2) dieser Modelle entstammen Vorarbeiten, die im Rahmen eines Praktikums bei der Firma QuantCo durchgeführt wurden [Fue24]. Der vollständige Quellcode beider Implementierungen befindet sich in Anhang A1.2.

### 4.2.1 SumBERT (Early Fusion)

SumBERT führt die Tabelleninformation bereits vor dem Encoder in das Modell ein. Die vorverarbeiteten Tabellendaten, repräsentiert als Merkmalsvektor  $\mathbf{x}_{\text{tab}} \in \mathbb{R}^F$ , werden zunächst per linearer Projektion (realisiert durch ein MLP) in den Vektorraum des Sprachmodells projiziert. Die Dimension der Projektion entspricht der Hidden-Size des Modells ( $H = 768$ ). Dieser projizierte Vektor wird anschließend elementweise zum Embedding des [CLS]-Tokens addiert, bevor die Sequenz den ersten Transformer-Layer durchläuft (siehe Abbildung 1).

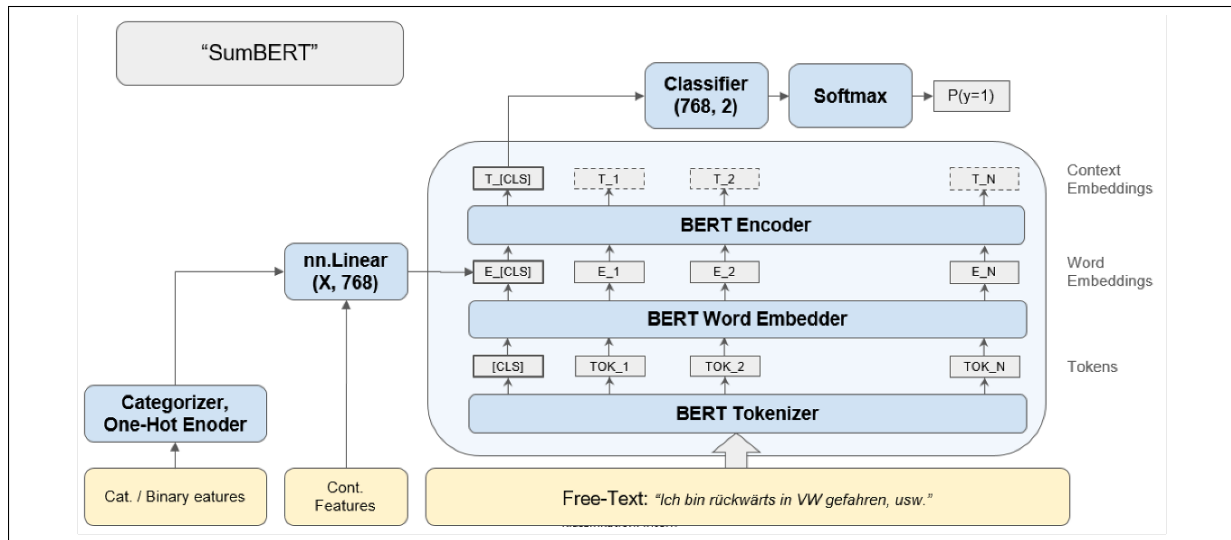


Abb. 1: Architektur von SumBERT: Addition der Tabellen-Repräsentation zum CLS-Token-Embedding vor dem Encoder. Quelle: [Fue24].

Formal lässt sich die Berechnung des fusionierten Eingabevektors  $\mathbf{e}_{\text{fused}}^{[\text{CLS}]}$  wie folgt beschreiben:

$$\mathbf{e}_{\text{fused}}^{[\text{CLS}]} = \mathbf{e}_{\text{text}}^{[\text{CLS}]} + \text{MLP}_{\text{tab}}(\mathbf{x}_{\text{tab}}) \quad (4.1)$$

Hierbei bezeichnet  $\mathbf{e}_{\text{text}}^{[\text{CLS}]} \in \mathbb{R}^H$  das ursprüngliche Embedding des [CLS]-Tokens (Eingabe-Ebene) und  $\mathbf{x}_{\text{tab}}$  den Vektor der tabellarischen Features. Durch diese Addition trägt das [CLS]-

Token von Beginn an sowohl Text- als auch Tabelleninformationen und gibt diese über die Self-Attention im Encoder an tiefere Schichten weiter.

#### 4.2.2 ConcatBERT (Late Fusion)

Im Gegensatz dazu nutzt ConcatBERT eine späte Fusion (Late Fusion). Text- und Tabellendaten werden hierbei zunächst separat verarbeitet. Der Text durchläuft den vollständigen BERT-Encoder, um eine kontextualisierte Repräsentation des [CLS]-Tokens zu generieren, die im Folgenden als  $\mathbf{h}_{\text{text}}^{[\text{CLS}]}$  bezeichnet wird. Wir verwenden hier  $\mathbf{h}$  (statt  $\mathbf{e}$ ), um zu kennzeichnen, dass es sich um einen Hidden State der Ausgabe-Ebene handelt. Erst im Anschluss an den Encoding-Prozess wird dieser Vektor mit den vorverarbeiteten Tabellen-Features  $\mathbf{x}_{\text{tab}}$  konkateniert (siehe Abbildung 2).

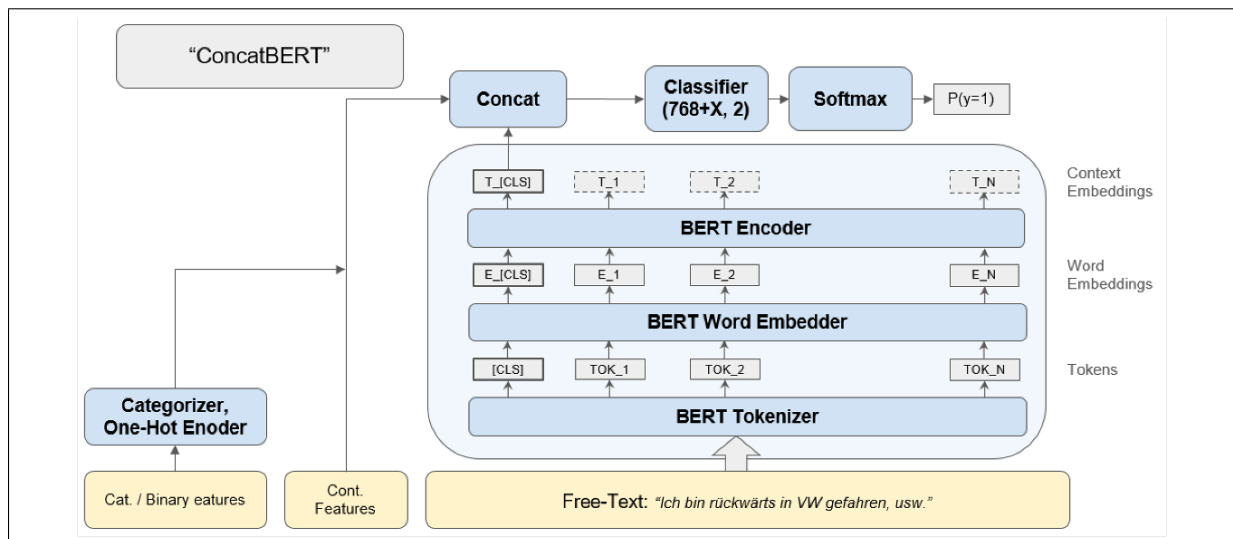


Abb. 2: Architektur von ConcatBERT: Konkatenation der Tabellen-Features mit dem Output des BERT-Encoders. Quelle: [Fue24].

Der resultierende Vektor  $\mathbf{h}_{\text{fusion}}$  dient als Eingabe für den Klassifikations-Head:

$$\mathbf{h}_{\text{fusion}} = [\mathbf{h}_{\text{text}}^{[\text{CLS}]} \parallel \mathbf{x}_{\text{tab}}] \quad (4.2)$$

Diese Architektur ist rechnerisch effizient und leicht zu implementieren, führt aber dazu, dass die Tabellendaten nicht mit dem Self-Attention-Mechanismus des Transformers interagieren können.

### 4.3 CrossBERT-Architektur

Die CrossBERT-Architektur erweitert einen BERT-Klassifikator um Cross-Attention zwischen Text und Tabellendaten. Anders als bei SumBERT und ConcatBERT (Abschnitt 4.2) werden Tabellendaten nicht nur als statischer Zusatzvektor verwendet, sondern als kurze Token-Sequenz in den Hidden-Space von BERT projiziert, die der Text über Cross-Attention auslesen kann (vgl. Abschnitt 2.2). Der vollständige Quellcode ist in Anhang A1.3 dokumentiert.

#### 4.3.1 Architekturüberblick

Abbildung 3 zeigt den Gesamtaufbau. Text wird mit `deepset/gbert-base` in eine Sequenz von Embeddings überführt und durch den Encoder verarbeitet. Die Dimension der Hidden-States beträgt dabei  $H = 768$ . Parallel dazu werden die tabellarischen Features durch einen *TabularTokenizer* in eine Sequenz von  $N$  Vektoren, die sogenannten *Tab Tokens*, kodiert. Kern der Architektur ist die Fusion über Cross-Attention-Blöcke. Diese verknüpfen gezielt das `[CLS]`-Token des Textes mit der Sequenz der Tab Tokens. Dies kann an zwei Stellen erfolgen: (i) *Early Fusion* direkt auf den Eingangs-Embeddings vor dem Encoder oder (ii) *Late Fusion* auf dem finalen Status nach dem Encoder. Der Klassifikationskopf (Classification Head) arbeitet anschließend auf dem fusionierten `[CLS]`-Vektor.

#### 4.3.2 TabularTokenizer

Die tabellarische Eingabe liegt nach der Vorverarbeitung für einen Batch der Größe  $B$  als Matrix  $\mathbf{X}_{\text{tab}} \in \mathbb{R}^{B \times F}$  vor, wobei  $F$  die Anzahl der Features (normalisierte numerische und One-Hot-kodierte kategoriale Variablen) bezeichnet.

Analog zur in Abschnitt 2.1.3 beschriebenen globalen Projektion werden aus diesen Features  $N$  latente Tokens generiert. Das Ergebnis ist ein Tensor  $\mathbf{E}_{\text{tab}} \in \mathbb{R}^{B \times N \times H}$ , der eine Sequenz von  $N$  Vektoren der Dimension  $H$  darstellt (siehe Abbildung 4a):

$$\mathbf{e}_i = \text{MLP}_i(\mathbf{X}_{\text{tab}}) \quad \text{für } i = 1, \dots, N \quad (4.3)$$

Die Gesamtsequenz ergibt sich durch Konkatenation der einzelnen Projektionen:

$$\mathbf{E}_{\text{tab}} = [\mathbf{e}_1, \dots, \mathbf{e}_N] \quad (4.4)$$

Technisch wird dies durch  $N$  parallele, lernbare MLP-Projektionsköpfe realisiert. Jeder Kopf bildet den gesamten Feature-Vektor  $\mathbf{X}_{\text{tab}}$  auf einen eigenen Token-Vektor  $\mathbf{e}_i$  ab. Die  $N$  Projektionsköpfe erzeugen somit  $N$  unterschiedliche, gelernte „Ansichten“ auf dieselbe Tabellenzeile.

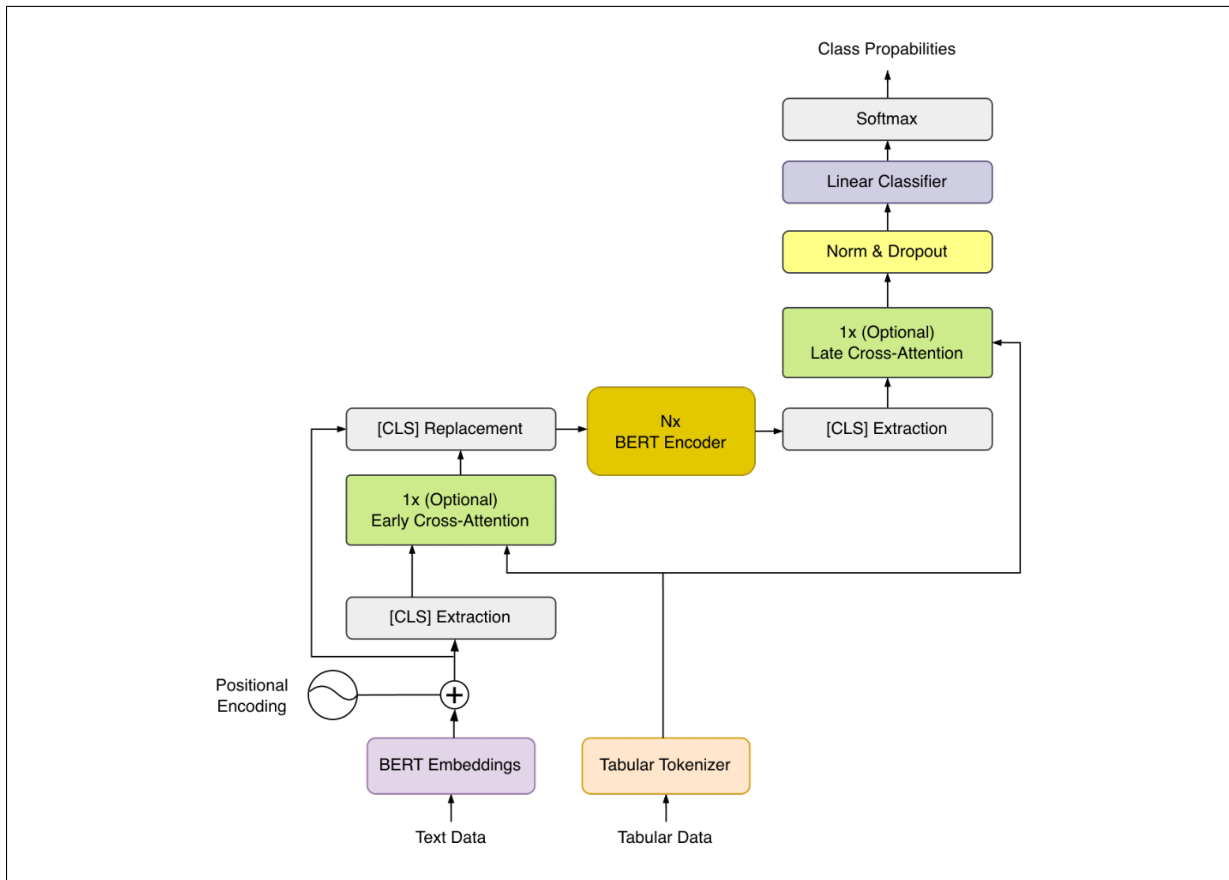


Abb. 3: Architektur von CrossBERT mit optionaler Early- und Late-Cross-Attention über Tab Tokens.

Diese globale Projektion entkoppelt die Sequenzlänge von der Eingabedimension  $F$ , die durch One-Hot-Encoding stark anwächst. Während eine direkte Tokenisierung aller Spalten oder eine Gruppierung nach Originalfeatures (wie beim später vorgestellten *TabTokenizer V2*) zu variablen Sequenzlängen führt, garantiert die fixe Anzahl  $N$  (Hyperparameter `num_tab_token`) eine konstante Recheneffizienz. Dies zwingt das Modell, die relevantesten Informationen der hochdimensionalen Tabellendaten in eine kompakte latente Repräsentation zu komprimieren.

### 4.3.3 Cross-Attention-Block

Die Fusion der Modalitäten erfolgt über einen Cross-Attention-Mechanismus (siehe Abschnitt 2.2), der gezielt in den Datenfluss des Transformers integriert ist. Wie in Abbildung 3 dargestellt, beginnt der Prozess mit der **CLS Extraction**, bei der das `[CLS]`-Token aus der aktuellen Sequenz (entweder Embedding oder Hidden State) herausgelöst wird. Dieses extrahierte Token fungiert als *Query*, während die erzeugten Tab Tokens als *Keys* und *Values* dienen. Nach der Anreicherung durch die Attention-Operation wird das aktualisierte Token mittels **CLS Replacement** an seine ursprüngliche Position in die Sequenz zurückgeschrieben, um den weiteren Fluss im Modell zu durchlaufen.

Sei  $\mathbf{c} \in \mathbb{R}^{B \times 1 \times H}$  der Vektor des [CLS]-Tokens (abhängig von der Position der Fusion entweder das initiale Embedding oder der Encoder-Output) und  $\mathbf{E}_{\text{tab}} \in \mathbb{R}^{B \times N \times H}$  die Sequenz der Tab Tokens. Die Projektionen für den Attention-Mechanismus berechnen sich wie folgt:

$$\mathbf{Q} = \mathbf{c}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{E}_{\text{tab}}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{E}_{\text{tab}}\mathbf{W}_V \quad (4.5)$$

Hierbei sind  $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{H \times H}$  die lernbaren Gewichtsmatrizen. Über die Attention-Gewichte lernt das Modell dynamisch, welche Informationen aus den Tabellendaten für die Interpretation des aktuellen Textes relevant sind. Die [CLS]-Repräsentation wird so kontextabhängig um tabellarische Informationen angereichert.

Die CrossBERT-Implementierung nutzt Multi-Head Attention und ergänzt diese um Residualpfade sowie ein Feed-Forward-Netzwerk (Abbildung 4b). Optional kann eine Tab-Maske genutzt werden, um einzelne Tab Tokens in der Attention auszublenden.

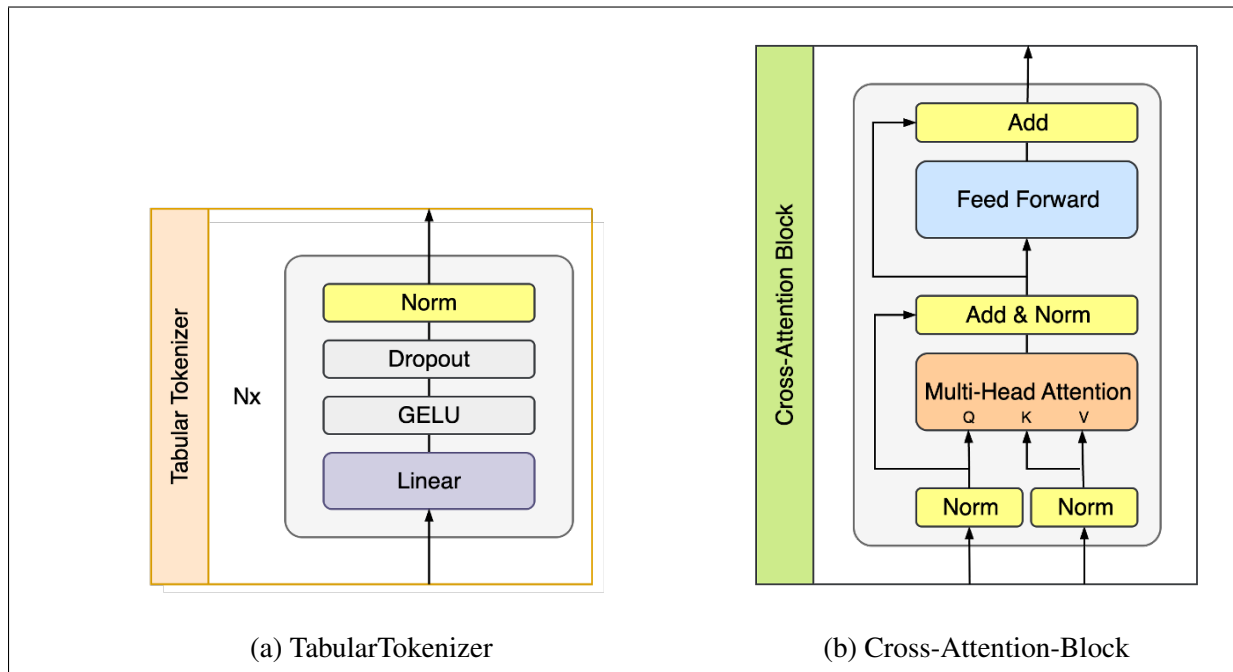


Abb. 4: Komponenten der CrossBERT-Architektur: Links die Projektion der Features in Tab Tokens, rechts deren Integration via Cross-Attention.

#### 4.3.4 Fusionsvarianten: Early, Late und Hybrid

Die Position des Cross-Attention-Blocks legt fest, ob die Fusion früh (vor dem Encoder), spät (nach dem Encoder) oder an beiden Stellen erfolgt:

- **Early Fusion:** Cross-Attention wird auf den [CLS]-Embedding-Vektor vor dem Encoder angewandt, sodass tabellarische Information die gesamte nachfolgende Kontextualisierung beeinflussen kann.
- **Late Fusion:** Cross-Attention wird erst auf den finalen Encoder- [CLS]-State angewandt, wodurch tabellarische Information als kontextabhängige Nachschärfung der Dokumentrepräsentation integriert wird.
- **Hybrid Fusion:** Early und Late Fusion werden kombiniert, um sowohl die Encoder-Features als auch die finale Entscheidungsschicht mit Tabellensignalen zu versorgen.

### 4.4 Trainings-Setup und Evaluationsmetriken

Damit die Modelle vergleichbar sind, werden alle Experimente mit einer gemeinsamen Pipeline und einer zentralen Konfiguration durchgeführt. Baselines und CrossBERT teilen sich dabei dieselben Trainingsparameter; sie unterscheiden sich nur in der Art der multimodalen Fusion (vgl. Abschnitt 4.3).

**Modell und Lernstrategie.** Als Textencoder wird `deepset/gbert-base` verwendet [CSM20]. GBERT ist ein vortrainiertes deutsches Sprachmodell auf Basis von BERT [DCLT19] und dient als Initialisierung für das end-to-end Fine-Tuning im jeweiligen Klassifikations-Setup. Als Verlustfunktion für das Training dient Cross-Entropy-Loss.

**Optimierung und Trainingsprotokoll.** Die zentralen Trainingshyperparameter sind in Tabelle 2 zusammengefasst. Zur Stabilisierung des Fine-Tunings wird eine Lernratenstrategie aus linearem Warmup und anschließendem Cosine Scheduling eingesetzt. Das Early Stopping überwacht `val/average_precision`. Die Validierung wird in festen Intervallen innerhalb einer Epoche ausgeführt (`val_check_interval=500`), sodass alle 500 Batches geprüft wird, ob das Early Stopping greifen soll. Für die *Gating*-Ablation nach Flamingo ([ADL<sup>+</sup>22], siehe Abschnitt 4.5) werden zusätzlich (i) der BERT-Encoder in den ersten zwei Epochen eingefroren (`freeze_bert_epochs=2`), (ii) die Gating-Parameter (`alpha_`) mit einer fünffach erhöhten Lernrate ohne Weight Decay trainiert (`lr_alpha_multiplier=5,0`) und (iii) die Epochenanzahl auf 2–6 begrenzt.

Tab. 2: Zentrale Trainingshyperparameter (Basiskonfiguration ohne Gating).

| Parameter                 | Wert                                      |
|---------------------------|-------------------------------------------|
| Optimizer                 | AdamW                                     |
| Lernrate                  | $1 \cdot 10^{-5}$                         |
| LR-Scheduler              | Linear Warmup (0,1) + Cosine              |
| Batchgröße (Train / Eval) | 64 / 128                                  |
| Epochen (min / max)       | 1 / 4                                     |
| Early Stopping            | Patience 5, Monitor val/average_precision |
| Validierungsanteil        | validation_fraction=0,2                   |

**Validierung, Hardware und Mixed Precision.** Die Validierung erfolgt auf einem festen Train/Val/Test-Split (vgl. Abschnitt 4.1). Alle Experimente laufen auf einer NVIDIA Tesla V100 (32 GB). Mixed Precision reduziert dabei Rechenzeit und Speicherbedarf.

**Evaluationsmetriken.** Als Hauptkennzahlen werden Average Precision (AP) und die Area Under the Receiver Operating Characteristic Curve (AUC-ROC) genutzt. AP ist bei unausgeglichene Klassen hilfreich, da sie die gesamte Precision-Recall-Kurve berücksichtigt. AUC-ROC misst, wie gut das Modell positive gegenüber negativen Beispielen trennt, unabhängig von einem festen Schwellenwert. In Multi-Class-Settings werden beide Kennzahlen one-vs-rest pro Klasse berechnet. Zusätzlich werden die Klassen binarisiert (positiv vs. Klasse 0), um eine konsistente Vergleichbarkeit über Datensätze hinweg zu gewährleisten. Zur Beurteilung der Modelleffizienz werden zudem bei jedem Experiment die Anzahl der trainierbaren Parameter und die Floating Point Operations (FLOPs) protokolliert.

## 4.5 Ablationsstudien

Um den Einfluss einzelner Architektur-Entscheidungen systematisch zu verstehen, werden gezielte Ablationsstudien durchgeführt.

Tab. 3: Übersicht der geplanten Ablationsstudien

| Ablation        | Variable (Wertebereich)                           | Ziel                                                                                           |
|-----------------|---------------------------------------------------|------------------------------------------------------------------------------------------------|
| Cross-Attention | Position: Early / Late / Hybrid                   | Einfluss des Fusionsorts (vor/nach Encoder vs. kombiniert)                                     |
| Tab Tokens      | $n_{\text{tab}}$ (Hybrid): 0 / 1 / 2 / 4 / 8 / 16 | Quantitativ prüfen, wie stark $n_{\text{tab}}$ die Leistung von CrossBERT (Hybrid) beeinflusst |
| Gating Layer    | Flamingo-style [ADL <sup>+</sup> 22]: Mit / Ohne  | Adaptive Gewichtung Text $\leftrightarrow$ Tab für stabilere Scores und ökonomische Kennzahlen |
| Tokenizer       | TabTokenizer: V1 / V2 (per-Feature-Tokenisierung) | Prüfen, ob Tokenisierung pro Feature die Fusion und Interpretierbarkeit verbessert             |

Im Fokus stehen (i) die Positionierung der Cross-Attention (Early/Late/Hybrid) und (ii) die Sensitivität gegenüber der Anzahl der Tab Tokens  $n_{\text{tab}}$  im Hybrid-Setup. Darüber hinaus werden zwei methodische Erweiterungen untersucht:

**Gated Cross-Attention.** Zur Stabilisierung der Fusion wird ein Gating-Mechanismus nach dem Vorbild von Flamingo [ADL<sup>+</sup>22] eingeführt. Hierbei wird der Output der Cross-Attention-Schicht nicht direkt addiert, sondern durch einen lernbaren Skalar  $\tanh(\alpha)$  gewichtet:  $\mathbf{y} = \mathbf{x} + \tanh(\alpha) \cdot \text{Attention}(\mathbf{x}, \mathbf{E}_{\text{tab}})$ . Da  $\alpha$  mit 0 initialisiert wird, startet das Modell als reiner Text-Klassifikator und lernt schrittweise, tabellarische Informationen hinzuzufügen. Für diese Ablation wurden die im Flamingo-Ansatz beschriebenen trainingsseitigen Anpassungen übernommen (Encoder-Freezing und erhöhte Lernrate für die Gate-Parameter; vgl. Abschnitt 4.4), um die Stabilität zu gewährleisten.

**TabTokenizer V2 (Per-Feature-Tokenisierung).** Während der Standard-Tokenizer die gesamte Tabellenzeile global auf  $N$  latente Tokens projiziert, ordnet der *TabTokenizer V2* jedem semantischen Feature (z. B. einer numerischen Variable oder einer Gruppe von One-Hot-kodierten Spalten) genau ein eigenes Token zu. Entscheidend ist hierbei, dass über eine *Feature Map* zusammengehörige One-Hot-Spalten aggregiert werden, sodass die Anzahl der erzeugten Tokens exakt der Anzahl der ursprünglichen Features im Rohdatensatz entspricht (und nicht der Dimension der expandierten Matrix  $\mathbf{X}_{\text{tab}}$ ). Jedes Feature erhält dazu einen eigenen, separaten MLP-Projektionskopf. Dies erhöht die Interpretierbarkeit, da jedes Element der Token-Sequenz direkt auf ein spezifisches Eingabefeature zurückzuführen ist.

## **5 Ergebnisse**

TODO ...

## **6 Diskussion und Fazit**

TODO ...

## Literaturverzeichnis

- [ADL<sup>+</sup>22] ALAYRAC, Jean-Baptiste ; DONAHUE, Jeff ; LUC, Pauline ; MIECH, Antoine ; BARR, Iain ; HASSON, Yana ; LENC, Karel ; MENSCH, Arthur ; MILLICAN, Katie ; REYNOLDS, Malcolm ; RING, Roman ; RUTHERFORD, Eliza ; CABI, Serkan ; HAN, Tengda ; GONG, Zhitao ; SAMANGOOEI, Sina ; MONTEIRO, Marianne ; MENICK, Jacob ; BORGEAUD, Sebastian ; BROCK, Andrew ; NEMATZADEH, Aida ; SHARIFZADEH, Sahand ; BINKOWSKI, Mikolaj ; BARREIRA, Ricardo ; VINYALS, Oriol ; ZISSERMAN, Andrew ; SIMONYAN, Karen: Flamingo: a Visual Language Model for Few-Shot Learning. In: *Advances in Neural Information Processing Systems* Bd. 35, 2022, S. 23716–23736
  
- [AU22] ABUBAKAR, H. D. ; UMAR, M.: Sentiment Classification: Review of Text Vectorization Methods: Bag of Words, Tf-Idf, Word2vec and Doc2vec. In: *SLUJST* 4 (2022), Aug, Nr. 1 & 2, S. 27–33. <http://dx.doi.org/10.56471/slujst.v4i.266>. – DOI 10.56471/slujst.v4i.266
  
- [B<sup>+</sup>24] BONNIER, N. u. a.: Benchmarking Multimodal Transformers. In: *arXiv preprint arXiv:2405.12345* (2024)
  
- [BAM19] BALTRUSAITIS, Tadas ; AHUJA, Chaitanya ; MORENCY, Louis-Philippe: Multimodal Machine Learning: A Survey and Taxonomy. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 41 (2019), Nr. 2, S. 423–443
  
- [BLS<sup>+</sup>24] BORISOV, V. ; LEEMANN, T. ; SESSLER, K. ; HAUG, J. ; PAWELCZYK, M. ; KASNECI, G.: Deep Neural Networks and Tabular Data: A Survey. In: *IEEE Trans. Neural Netw. Learning Syst.* 35 (2024), Jun, Nr. 6, S. 7499–7519. <http://dx.doi.org/10.1109/TNNLS.2022.3229161>. – DOI 10.1109/TNNLS.2022.3229161
  
- [BSP23] BADARO, G. ; SAEED, M. ; PAPOTTI, P.: Transformers for Tabular Data Representation: A Survey of Models and Applications. In: *Transactions of the Association for Computational Linguistics* 11 (2023), S. 227–249. [http://dx.doi.org/10.1162/tacl\\_a\\_00544](http://dx.doi.org/10.1162/tacl_a_00544). – DOI 10.1162/tacl\_a\_00544
  
- [C<sup>+</sup>23] CHEN, Xi u. a.: PaLI: A Jointly-Scaled Multilingual Language-Image Model. In: *ICLR*, 2023
  
- [CSM20] CHAN, B. ; SCHWETER, S. ; MÖLLER, T.: German’s Next Language Model. (2020), Dec. <http://dx.doi.org/10.48550/arXiv.2010.10906>. – DOI 10.48550/arXiv.2010.10906

- [DBK<sup>+</sup>21] DOSOVITSKIY, Alexey ; BEYER, Lucas ; KOLESNIKOV, Alexander ; WEISSENBORN, Dirk ; ZHAI, Xiaohua ; UNTERTHINER, Thomas ; DEHGHANI, Mostafa ; MINDERER, Matthias ; HEIGOLD, Georg ; GELLY, Sylvain ; USZKOREIT, Jakob ; HOULSBY, Neil: An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In: *International Conference on Learning Representations*, 2021
- [DCLT19] DEVLIN, Jacob ; CHANG, Ming-Wei ; LEE, Kenton ; TOUTANOVA, Kristina: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. (2019), May. <http://dx.doi.org/10.48550/arXiv.1810.04805>. – DOI 10.48550/arXiv.1810.04805
- [Fue24] FUEST, Johannes: *Subrogation (Regress) @ HUK - Internship Presentation*. Aug 2024. – Interne Präsentation bei QuantCo (unveröffentlicht), Zugriff: 12.01.2026
- [GB21] GU, K. ; BUDHKAR, A.: A Package for Learning on Tabular and Text Data with Transformers. In: ZADEH, A. (Hrsg.) ; MORENCY, L.-P. (Hrsg.) ; LIANG, P. P. (Hrsg.) ; ROSS, C. (Hrsg.) ; SALAKHUTDINOV, R. (Hrsg.) ; PORIA, S. (Hrsg.) ; CAMBRIA, E. (Hrsg.) ; SHI, K. (Hrsg.): *Proceedings of the Third Workshop on Multimodal Artificial Intelligence*. Mexico City, Mexico : Association for Computational Linguistics, Jun 2021, S. 69–73
- [GRKB21] GORISHNIY, Y. ; RUBACHEV, I. ; KHRULKOV, V. ; BABENKO, A.: Revisiting Deep Learning Models for Tabular Data. In: *Advances in Neural Information Processing Systems* (2021)
- [HKCK20] HUANG, X. ; KHETAN, A. ; CVITKOVIC, M. ; KARNIN, Z.: TabTransformer: Tabular Data Modeling Using Contextual Embeddings. (2020), Dec. <http://dx.doi.org/10.48550/arXiv.2012.06678>. – DOI 10.48550/arXiv.2012.06678
- [J<sup>+</sup>24] JIAO, Q. u. a.: Recent Advances in Multimodal Fusion. In: *arXiv preprint arXiv:2401.01234* (2024)
- [Kag18] KAGGLE: *Women's E-Commerce Clothing Reviews*. <https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews>. Version: 2018. – Zugriff: 2024-01-12
- [Kag19] KAGGLE: *PetFinder.my Adoption Prediction*. <https://www.kaggle.com/c/petfinder-adoption-prediction>. Version: 2019. – Zugriff: 2024-01-12
- [L<sup>+</sup>24] LI, Y. u. a.: Multimodal Learning with Transformers: A Survey. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2024)

- [MCCD13] MIKOLOV, Tomas ; CHEN, Kai ; CORRADO, Greg ; DEAN, Jeffrey: Efficient Estimation of Word Representations in Vector Space. (2013), Sep. <http://dx.doi.org/10.48550/arXiv.1301.3781>. – DOI 10.48550/arXiv.1301.3781
  
- [PNI<sup>+</sup>18] PETERS, Matthew E. ; NEUMANN, Mark ; IYYER, Mohit ; GARDNER, Matt ; CLARK, Christopher ; LEE, Kenton ; ZETTLEMOYER, Luke: Deep Contextualized Word Representations. In: WALKER, Marilyn (Hrsg.) ; JI, Heng (Hrsg.) ; STENT, Amanda (Hrsg.): *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. New Orleans, Louisiana : Association for Computational Linguistics, Jun 2018, S. 2227–2237
  
- [SB88] SALTON, Gerard ; BUCKLEY, Chris: Term-weighting approaches in automatic text retrieval. In: *Information Processing & Management* 24 (1988), Nr. 5, S. 513–523. [http://dx.doi.org/10.1016/0306-4573\(88\)90021-0](http://dx.doi.org/10.1016/0306-4573(88)90021-0). – DOI 10.1016/0306-4573(88)90021-0
  
- [SGS<sup>+</sup>21] SOMEPALI, G. ; GOLDBLUM, M. ; SCHWARZSCHILD, A. ; BRUSS, C. B. ; GOLDSTEIN, T.: SAINT: Improved Neural Networks for Tabular Data via Row Attention and Contrastive Pre-Training. In: *arXiv preprint arXiv:2106.01342* (2021). <https://arxiv.org/abs/2106.01342v1>
  
- [TB19] TAN, H. ; BANSAL, M.: LXMERT: Learning Cross-Modality Encoder Representations from Transformers. (2019), Dec. <http://dx.doi.org/10.48550/arXiv.1908.07490>. – DOI 10.48550/arXiv.1908.07490
  
- [VSP<sup>+</sup>17] VASWANI, Ashish ; SHAZEER, Noam ; PARMAR, Niki ; USZKOREIT, Jakob ; JONES, Llion ; GOMEZ, Aidan N. ; KAISER, Łukasz ; POLOSUKHIN, Illia: Attention is All you Need. In: *Advances in Neural Information Processing Systems* 30, 2017
  
- [YNYR20] YIN, Pengcheng ; NEUBIG, Graham ; YIH, Wen-tau ; RIEDEL, Sebastian: TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, S. 8413–8426



## A1 Anhang

### A1.1 Modellarchitektur CrossFitter

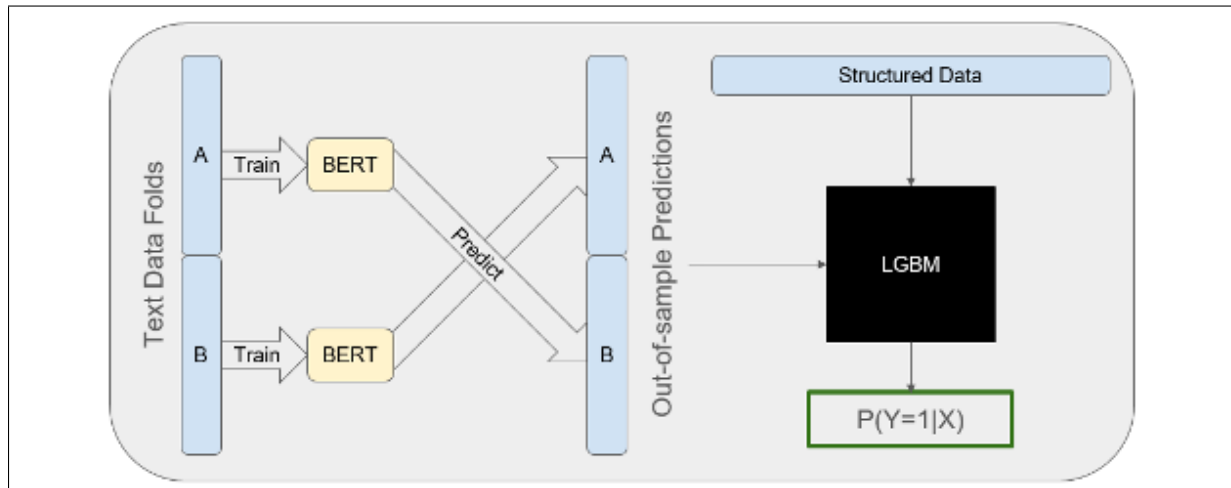


Abb. 5: Schematische Darstellung der CrossFitter-Modellarchitektur im produktiven Einsatz.  
Quelle: [Fue24].

### A1.2 Quellcode der Baseline-Modelle

Der folgende Quellcode zeigt die PyTorch-Implementierung der verwendeten Baseline-Modelle *ConcatBERT* und *SumBERT*. Beide Klassen erben von `BertForSequenceClassification` und passen den Forward-Pass entsprechend der jeweiligen Fusionsstrategie an.

```
class ConcatBert (BertForSequenceClassification):
    """Custom BERT that handles numerical and one-hot encoded categorical
        data by concatenating it to the output embedding
        of the CLS token.

    Attributes:
    -----
    classifier: ClassificationHead
        A classification module that is identical to the one in
        BertForSequenceClassification, except for added input dimensions
        from the numerical data.

    Methods:
    -----
    forward(...):
        Takes in input ids, attention mask, and a tensor of extra data to
        compute the binary output. Input ids and attention mask are
        first fed through the base bert model.
```

```
    The resulting CLS context embedding is then concatenated to the
    extra data tensor and fed through the classificationHead to get
    the final output.
"""

def __init__(self, cfg: BertConfig, extra_data_dim: int, hidden_dim:
int):
    super().__init__(cfg)

class ClassificationHead(nn.Module):
    """Head for sentence-level classification tasks."""

    def __init__(self, cfg, extra_data_dim, hidden_dim):
        super().__init__()
        total_dims = cfg.hidden_size + extra_data_dim
        assert (
            hidden_dim <= total_dims
        ), "The hidden dim should be less or equal than the input
        embedding"
        self.dense = nn.Linear(total_dims, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        classifier_dropout = (
            cfg.classifier_dropout
            if cfg.classifier_dropout is not None
            else cfg.hidden_dropout_prob
        )
        self.dropout = nn.Dropout(classifier_dropout)
        self.out_proj = nn.Linear(hidden_dim, cfg.num_labels)

    def forward(self, features, **kwargs):
        x = self.dropout(features)
        x = self.dense(x)
        x = self.bn1(x)
        x = torch.relu(x)
        x = self.dropout(x)
        x = self.out_proj(x)
        return x

self.classifier = ClassificationHead(cfg, extra_data_dim,
hidden_dim)
# https://tfjs/web/DefaultCollection/GIT\_Projects/\_git/hf4-regress-
exploration/pullRequest/81216#1737385342
self.post_init()

def forward(
    self,
```

```
input_ids: torch.Tensor | None = None,
attention_mask: torch.Tensor | None = None,
token_type_ids: torch.Tensor | None = None,
position_ids: torch.Tensor | None = None,
head_mask: torch.Tensor | None = None,
inputs_embeds: torch.Tensor | None = None,
labels: torch.Tensor | None = None,
output_attentions: bool | None = None,
output_hidden_states: bool | None = None,
return_dict: bool | None = None,
extra_data: torch.Tensor | None = None,
) -> tuple | SequenceClassifierOutput:

    return_dict = (
        return_dict if return_dict is not None else self.config.
            use_return_dict
    )

    outputs = self.bert(
        input_ids,
        attention_mask=attention_mask,
        token_type_ids=token_type_ids,
        position_ids=position_ids,
        head_mask=head_mask,
        inputs_embeds=inputs_embeds,
        output_attentions=output_attentions,
        output_hidden_states=output_hidden_states,
        return_dict=return_dict,
    )

    # Using the pooler output as document embedding. This corresponds
    # to a post-processed version of the <CLS> token embedding.
    cls_embedding = outputs.pooler_output

    if extra_data is not None:
        output = torch.cat((cls_embedding, extra_data), dim=-1)
    else:
        raise ValueError("Must supply extra data!")
    logits = self.classifier(output)
    if not return_dict:
        output = (logits,) + outputs[2:]
        return output

    return SequenceClassifierOutput(
        logits=logits,
        hidden_states=outputs.hidden_states,
        attentions=outputs.attentions,
```

```
)

class SumBert(BertForSequenceClassification):
    """
    SumBert integrates additional feature-embeddings with a BERT base model
    used for a classification task.

    Parameters
    -----
    feature_layer:
        Linear layer used to add the numerical and categorical data to the
        word embedding of the CLS token.

    Methods
    -----
    forward(...):
        Gets word embeddings from input ids through bert. The adds the
        output of its feature layer to the CLS word embedding to insert
        numerical / categorical data.
        Subsequently uses BertModels forward function to get outputs.
    """

    def __init__(self, cfg: BertConfig, extra_data_dim: int):
        super().__init__(cfg)
        self.feature_layer = nn.Linear(extra_data_dim, 768)

    def forward(
        self,
        input_ids: torch.Tensor | None = None,
        attention_mask: torch.Tensor | None = None,
        token_type_ids: torch.Tensor | None = None,
        position_ids: torch.Tensor | None = None,
        head_mask: torch.Tensor | None = None,
        inputs_embeds: torch.Tensor | None = None,
        labels: torch.Tensor | None = None,
        output_attentions: bool | None = None,
        output_hidden_states: bool | None = None,
        return_dict: bool | None = None,
        extra_data: torch.Tensor | None = None,
    ):
        # Avoid performing computation under torch.no_grad() to ensure
        # word embeddings can be updated during training.
        text_embedding = self.bert.embeddings.word_embeddings(input_ids)
        text_embedding[:, 0] += self.feature_layer(extra_data)
        outputs = self.bert(
```

```
        inputs_embeds=text_embedding,
        attention_mask=attention_mask,
    )
    pooled_output = outputs.pooler_output
    pooled_output = self.dropout(pooled_output)
    logits = self.classifier(pooled_output)
    return SequenceClassifierOutput(
        logits=logits,
        hidden_states=outputs.hidden_states,
        attentions=outputs.attentions,
    )
```

Code 1: PyTorch-Implementierung von ConcatBERT und SumBERT

### A1.3 Quellcode der CrossBERT-Architektur

Der folgende Quellcode enthält die PyTorch-Implementierung der in Abschnitt 4.3 beschriebenen CrossBERT-Architektur, einschließlich *TabularTokenizer* und *CrossAttentionBlock*.

```
class CrossBert(BertForSequenceClassification):
    """BERT classifier with Early and Late cross-attention over tabular tokens."""

    def __init__(self, cfg: BertConfig, extra_data_dim: int, cross_attention_positions: Dict[
        str, bool],
        num_tab_tokens: int = 4, cross_attention_heads: Optional[int] = None):
        super().__init__(cfg)
        self.tabular_tokenizer = TabularTokenizer(num_features=extra_data_dim, hidden_size=cfg
            .hidden_size,
            num_tab_tokens=num_tab_tokens, dropout=cfg.
                hidden_dropout_prob)

        self.cross_attn_early = CrossAttentionBlock(embed_dim=768, num_heads=8) if
            cross_attention_positions.get("early") else None
        self.cross_attn_late = CrossAttentionBlock(embed_dim=768, num_heads=8) if
            cross_attention_positions.get("late") else None
        self.post_fusion_ln = nn.LayerNorm(cfg.hidden_size)
        self.post_fusion_dropout = nn.Dropout(cfg.hidden_dropout_prob)

    def forward(self, input_ids=None, attention_mask=None, extra_data=None, tab_mask=None, **
        kwargs):
        tab_tokens = self.tabular_tokenizer(extra_data) if extra_data is not None else None
        embeddings = self.bert.embeddings(input_ids=input_ids)

        if self.cross_attn_early and tab_tokens is not None:
            cls_early = self.cross_attn_early(embeddings[:, :1, :], tab_tokens, tab_mask=
                tab_mask)
            embeddings = torch.cat([cls_early, embeddings[:, 1:, :]], dim=1)

        encoder_outputs = self.bert.encoder(hidden_states=embeddings,
            attention_mask=self.bert.
                get_extended_attention_mask(attention_mask,
                    attention_mask.shape, attention_mask.device))
        cls_token = encoder_outputs.last_hidden_state[:, :1, :]
```

```
        if self.cross_attn_late and tab_tokens is not None:
            cls_token = self.cross_attn_late(cls_token, tab_tokens, tab_mask=tab_mask)

        fused_cls = self.post_fusion_dropout(self.post_fusion_ln(cls_token)).squeeze(1)
        return SequenceClassifierOutput(logits=self.classifier(fused_cls))

class CrossAttentionBlock(nn.Module):
    def __init__(self, embed_dim, num_heads, dim_feedforward=2048, dropout=0.1):
        super().__init__()
        self.attention = nn.MultiheadAttention(embed_dim, num_heads, dropout=dropout,
                                                batch_first=True)
        self.layer_norm_1 = nn.LayerNorm(embed_dim)
        self.ffn = nn.Sequential(nn.Linear(embed_dim, dim_feedforward), nn.GELU(), nn.Dropout(
            dropout), nn.Linear(dim_feedforward, embed_dim))
        self.layer_norm_2 = nn.LayerNorm(embed_dim)
        self.dropout_attn, self.dropout_ffn = nn.Dropout(dropout), nn.Dropout(dropout)

    def forward(self, cls_token, tab_tokens, tab_mask=None):
        q, k = self.layer_norm_1(cls_token), self.layer_norm_1(tab_tokens)
        attn_out, _ = self.attention(q, k, k, key_padding_mask=(tab_mask == 0) if tab_mask is
            not None else None)
        x = cls_token + self.dropout_attn(attn_out)
        return x + self.dropout_ffn(self.ffn(self.layer_norm_2(x)))

class TabularTokenizer(nn.Module):
    def __init__(self, num_features, hidden_size, num_tab_tokens=8, dropout=0.1):
        super().__init__()
        self.projections = nn.ModuleList([nn.Sequential(nn.Linear(num_features, hidden_size),
                                                         nn.GELU(),
                                                         nn.Dropout(dropout), nn.LayerNorm(
                                                             hidden_size)) for _ in range(
                                                             num_tab_tokens)])

    def forward(self, tabular_input):
        return torch.cat([proj(tabular_input).unsqueeze(1) for proj in self.projections], dim
                          =1)
```

Code 2: PyTorch-Implementierung von CrossBERT

**Persönliche Angaben / Personal details**

**Städler, Sebastian**

Familienname, Vorname / Surnames, given names

**19.03.2002**

Geburtsdatum / Date of birth

**Informatik**

Studiengang / Course of study

**00383022**

Matrikelnummer / Student registration number

**Eigenständigkeitserklärung*****Declaration***

Hiermit versichere ich, dass ich diese Arbeit selbständig verfasst und noch nicht anderweitig für Prüfungszwecke vorgelegt habe. Ich habe keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt. Die Arbeit wurde weder in Gänze noch in Teilen von einer Künstlichen Intelligenz (KI) erstellt, es sei denn, die zur Erstellung genutzte KI wurde von der zuständigen Prüfungskommission oder der bzw. dem zuständigen Prüfenden ausdrücklich zugelassen. Wörtliche oder sinngemäße Zitate habe ich als solche gekennzeichnet.

Es ist mir bekannt, dass im Rahmen der Beurteilung meiner Arbeit Plagiatserkennungssoftware zum Einsatz kommen kann.

Es ist mir bewusst, dass Verstöße gegen Prüfungsvorschriften zur Bewertung meiner Arbeit mit „nicht ausreichend“ und in schweren Fällen auch zum Verlust sämtlicher Wiederholungsversuche führen können.

*I hereby certify that I have written this thesis independently and have not submitted it elsewhere for examination purposes. I have not used any sources or aids other than those indicated. The work has not been created in whole or in part by an artificial intelligence (AI), unless the AI used to create the work has been expressly approved by the responsible examination board or examiner. I have marked verbatim quotations or quotations in the spirit of the text as such.*

*I am aware that plagiarism detection software may be used in the assessment of my work.*

*I am aware that violations of examination regulations can lead to my work being graded as "unsatisfactory" and, in serious cases, to the loss of all repeat attempts.*

\_\_\_\_\_  
Unterschrift Studierende/Studierender / Signature student

**96450 Coburg, den 13.01.2026**

Ort, Datum / Place, date